

LABORATORY OBSERVATION/RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Observation Task No.: 3

Student Name: V S SRAVAN M

Roll No.: A24126552268

Date: 16/8/2026

1. TITLE

Interactive To-Do List Application using HTML, CSS and JavaScript

2. AIM

To create a dynamic task management and to-do list web application supporting task creation, completion toggling, and task removal with event-driven DOM manipulation.

3. OBJECTIVE

The objectives of this experiment are:

- To design an intuitive, centered to-do list interface with input fields and action buttons.
 - To implement dynamic element creation using `document.createElement()` and `appendChild()`.
 - To handle event delegation for item click (toggle completed) and delete button triggers.
 - To apply CSS text-decoration and color transitions for completed items.
 - To validate against blank task submissions.
-

4. THEORY

The To-Do List application demonstrates the fundamental CRUD operations (Create, Read, Update, Delete) on the Document Object Model. JavaScript event listeners intercept user interactions, instantiate new DOM elements, attach delete callbacks, and append them into the list container while maintaining clean separation between structure, style, and behavior.

Theory of To-Do Application Architecture:

- **DOM Node Creation:** Dynamically constructing `<li>`, `<span>`, and `<button>` elements using `document.createElement()`.
 - **Event Delegation & Bubbling:** Attaching click handlers to manage strike-through toggling and element removal.
 - **Class List Toggling:** Using `classList.toggle('completed')` to toggle line-through styling.
 - **Input Sanitation:** Verifying `taskInput.value.trim()` is non-empty before adding tasks to prevent blank entries.
-

5. ALGORITHM / PROCEDURE

1. Set up project workspace folders: Observation/Observation task 3/Src, Documents, and Screenshots.

2. In index.html, construct header, input row with add button, and unordered list container (#taskList).
 3. In style.css, apply centered card container styling, modern buttons, and completed task styles.
 4. In script.js, handle click event on Add button and keypress on Enter key to append new task items.
 5. Attach click listener on task text to toggle completed state, and on delete button to remove task node.
 6. Open in Google Chrome and verify task creation, completion strikethrough, and item deletion.
-

6. PROGRAM

index.html (Lines 1-24)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<title>My To-Do List</title>
<link rel="stylesheet" href="style.css">
</head>
<body>

<div class="container">
  <h1>My To-Do List</h1>

  <div class="input-row">
    <input type="text" id="taskInput" placeholder="Enter a task...">
    <button id="addBtn">Add Task</button>
  </div>

  <ul id="taskList"></ul>
  <p id="emptyMsg">No tasks available.</p>
</div>

<script src="script.js"></script>
</body>
</html>
```

style.css (Lines 1-110)

```
body {
  font-family: Arial, sans-serif;
  background: #eef2f5;
  display: flex;
  justify-content: center;
  padding: 40px 15px;
}

.container {
  background: #fff;
  width: 100%;
  max-width: 420px;
  padding: 25px 30px;
  border-radius: 8px;
  box-shadow: 0 4px 14px rgba(0,0,0,0.15);
}

h1 {
  text-align: center;
  font-size: 22px;
  color: #2c3e50;
  margin-bottom: 20px;
}

.input-row {
  display: flex;
  gap: 8px;
  margin-bottom: 20px;
}

#taskInput {
  flex: 1;
  padding: 9px 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
  font-size: 14px;
}

#addBtn {
  padding: 9px 16px;
  background: #2c3e50;
  color: #fff;
  border: none;
  border-radius: 5px;
  cursor: pointer;
  font-size: 14px;
}

#addBtn:hover { background: #1a252f; }

#taskList {
  list-style: none;
  padding: 0;
  margin: 0;
}

#emptyMsg {
```

```
text-align: center;
color: #888;
font-size: 14px;
padding: 10px 0;
}

.task-item {
display: flex;
align-items: center;
justify-content: space-between;
background: #f8f9fa;
padding: 10px 12px;
border-radius: 5px;
margin-bottom: 8px;
border-left: 4px solid #2c3e50;
}

.task-text {
flex: 1;
font-size: 14px;
color: #333;
word-break: break-word;
margin-right: 10px;
}

.task-buttons {
display: flex;
gap: 6px;
}

.complete-btn, .delete-btn {
border: none;
padding: 6px 10px;
border-radius: 4px;
font-size: 12px;
cursor: pointer;
color: #fff;
}

.complete-btn { background: #27ae60; }
.complete-btn:hover { background: #1e8449; }
.delete-btn { background: #e74c3c; }
.delete-btn:hover { background: #c0392b; }

.completed .task-text {
text-decoration: line-through;
color: #999;
}

.completed {
border-left-color: #27ae60;
background: #eafaf1;
}
```

script.js (Lines 1-78)

```
// ---- DOM selection ----
const taskInput = document.getElementById('taskInput');
const addBtn = document.getElementById('addBtn');
const taskList = document.getElementById('taskList');
const emptyMsg = document.getElementById('emptyMsg');

// ---- Check and toggle empty message ----
function updateEmptyMessage() {
  if (taskList.children.length === 0) {
    emptyMsg.style.display = 'block';
  } else {
    emptyMsg.style.display = 'none';
  }
}

// ---- Add task ----
function addTask() {
  const taskText = taskInput.value.trim();

  if (taskText === '') {
    alert('Please enter a task before adding.');
```

```
    return;
  }

  // Create list item
  const li = document.createElement('li');
  li.className = 'task-item';

  // Task text span
  const span = document.createElement('span');
  span.className = 'task-text';
  span.textContent = taskText;

  // Buttons wrapper
  const btnWrapper = document.createElement('div');
  btnWrapper.className = 'task-buttons';

  // Complete button
  const completeBtn = document.createElement('button');
  completeBtn.className = 'complete-btn';
  completeBtn.textContent = 'Complete';
  completeBtn.addEventListener('click', function () {
    li.classList.toggle('completed');
  });

  // Delete button
  const deleteBtn = document.createElement('button');
  deleteBtn.className = 'delete-btn';
  deleteBtn.textContent = 'Delete';
  deleteBtn.addEventListener('click', function () {
    li.remove();
    updateEmptyMessage();
  });

  btnWrapper.appendChild(completeBtn);
  btnWrapper.appendChild(deleteBtn);
}
```

```
li.appendChild(span);
li.appendChild(btnWrapper);
taskList.appendChild(li);

taskInput.value = '';
taskInput.focus();

updateEmptyMessage();
}

// ---- Event listeners ----
addBtn.addEventListener('click', addTask);

taskInput.addEventListener('keypress', function (e) {
  if (e.key === 'Enter') {
    addTask();
  }
});

// ---- Initial state ----
updateEmptyMessage();
```

7. CODE EXPLANATION

DOM Method / Property	Explanation
<code>document.createElement('li')</code>	Dynamically creates a new list item element in memory.
<code>appendChild(node)</code>	Injects the newly created node as a child of the task list container.
<code>element.remove()</code>	Removes the selected task element directly from the DOM tree.
<code>classList.toggle('completed')</code>	Toggles the completion class to apply or remove strikethrough.
<code>.trim()</code>	Trims whitespace from task input string to prevent empty entries.
<code>addEventListener('keydown')</code>	Listens for the Enter key press to submit tasks without clicking the button.

8. TEST CASES AND EXECUTION DATA

The following test suite was executed in Google Chrome to verify to-do list CRUD functionalities:

TC ID	Test Description	Input / Action	Expected Result	Status
TC-01	Task Addition	Type 'Complete Web Lab Record' and click Add	New list item is instantiated and displayed in task list	Pass
TC-02	Empty Input Validation	Click Add with blank input field	Submission blocked; no empty task added	Pass
TC-03	Enter Key Submission	Type task and press Enter key	Task added to list automatically	Pass
TC-04	Task Completion Toggle	Click on task text	Task toggles completed style with strikethrough and gray text	Pass
TC-05	Task Deletion	Click delete button (X) next to task	Task item is completely removed from the DOM	Pass

9. OUTPUT

Output: Rendered To-Do List Application in Google Chrome

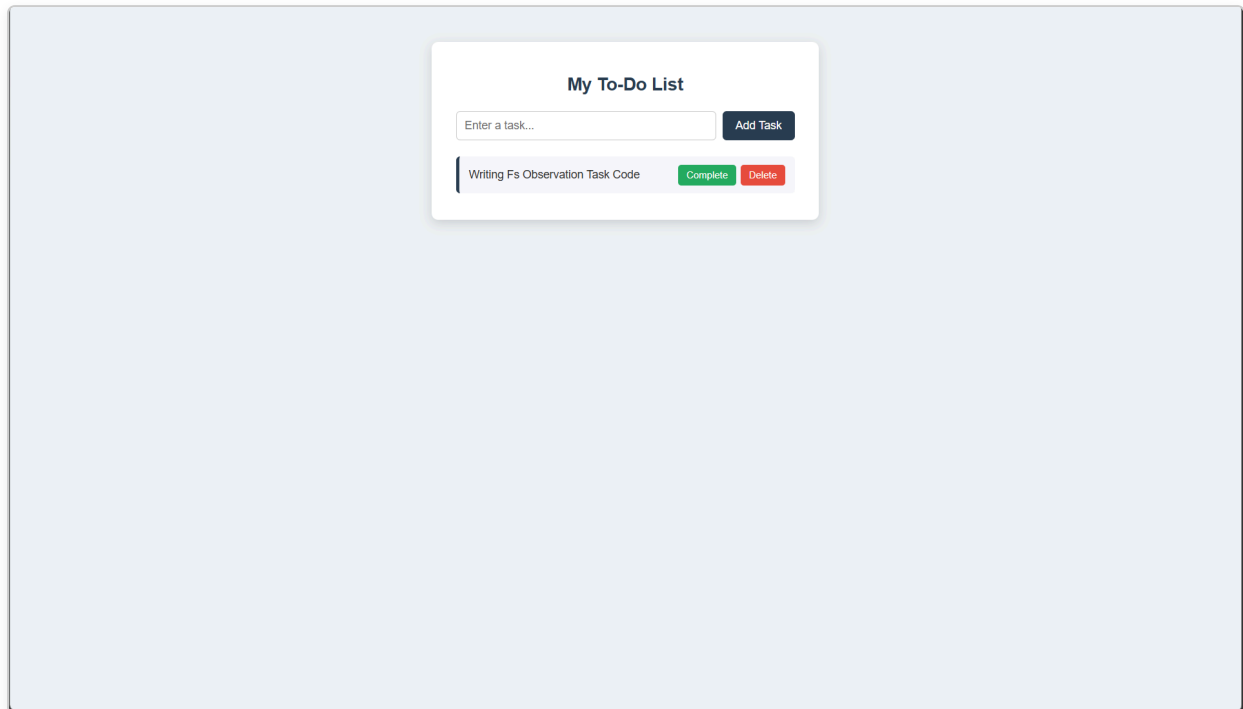


Figure 1: Initial to-do list with active tasks.

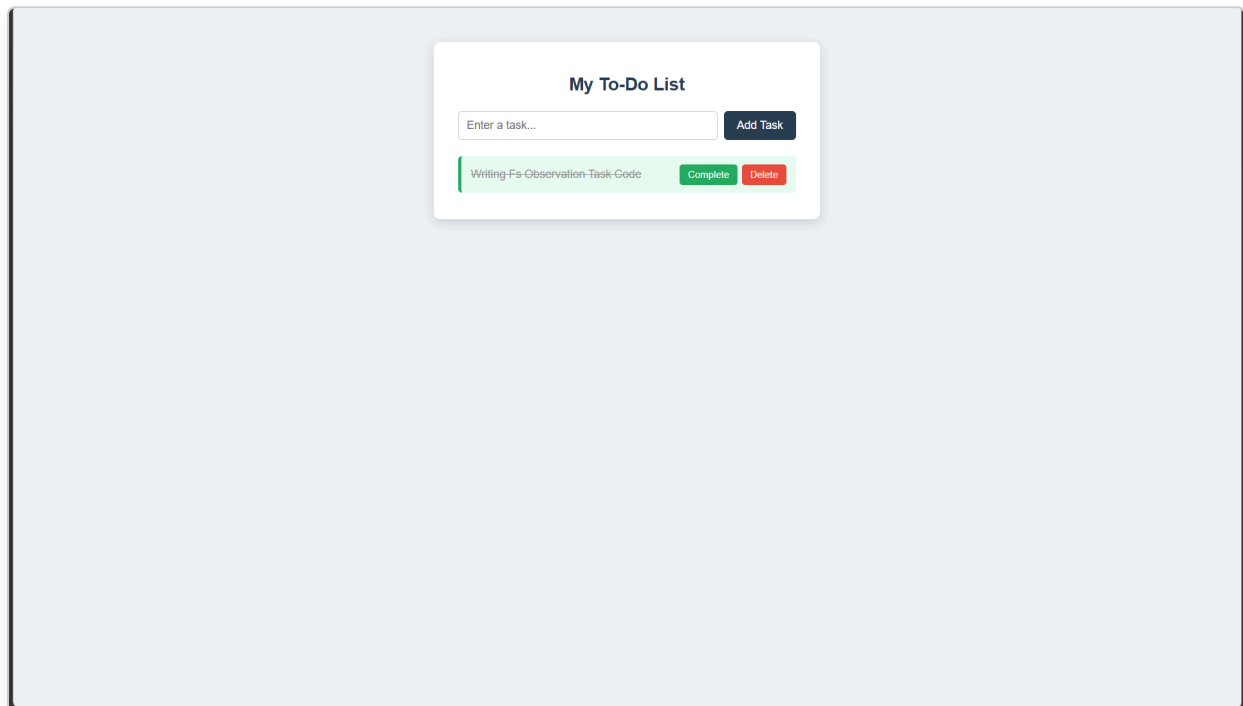


Figure 2: To-do list showing completed tasks with strikethrough and item deletion.

10. RESULT

Thus, the interactive To-Do List web application was successfully designed, implemented, and verified using manual test cases under browser defaults.

Submission Package Structure:

```
Observation/Observation task 3/
├── Documents/
│   ├── ObsWeek3.docx
│   └── ObsWeek3.pdf
├── Screenshots/
│   ├── Screenshot 2026-08-16 221211.png
│   └── Screenshot 2026-08-16 221223.png
└── Src/
    ├── index.html
    ├── script.js
    └── style.css
```